

# Lab 9: Implementation of the Codes for Developing GUI's in Java

## Objective of the Lab:

To introduce students to **Graphical User Interface (GUI)** development using **Java Swing**, helping them understand how to create simple interactive windows with buttons, text fields, and other components.

## What is GUI (Graphical User Interface)?

GUI stands for **Graphical User Interface**, which allows users to interact with programs through **visual elements** such as:

- Windows
- Buttons
- Text fields
- Checkboxes
- Dialog boxes

Instead of typing commands in (CLI), GUI makes programs **user-friendly** and easier to use

## Basic Java Swing Components You Must Know

| Component               | Description                                               |
|-------------------------|-----------------------------------------------------------|
| 1. <b>JFrame</b>        | Main window or screen of the app                          |
| 2. <b>JLabel</b>        | Displays text (non-editable)                              |
| 3. <b>TextField</b>     | Allows user to type input (one line)                      |
| 4. <b>PasswordField</b> | Input for passwords (hidden text)                         |
| 5. <b>Button</b>        | A button user can click to perform action                 |
| 6. <b>TextArea</b>      | Input box for multiple lines (like feedback form)         |
| 7. <b>CheckBox</b>      | Multiple options selection (can select more than one)     |
| 8. <b>RadioButton</b>   | Single option selection among many                        |
| 9. <b>ComboBox</b>      | Dropdown menu (select one item)                           |
| 10. <b>Panel</b>        | Grouping layout box helps organize GUI into sections      |
| 11. <b>OptionPane</b>   | Pops up small dialog box to show message or ask for input |

# How GUI Programs Work in Java (Step-by-Step)

1. **Import** Swing package using `import javax.swing.*;`
2. **Create a class** that extends **JFrame** or uses **JFrame** object.
3. **Add Components** like buttons, text fields, labels, etc.
4. **Add Action Listeners** to handle events like button clicks.
5. **Set Frame Properties** like size, title, visibility.

## Component 1: JFrame Main Window

### Purpose:

A JFrame is the **main window** where all other components (buttons, text fields, etc.) are added.

### Real-Life Example:

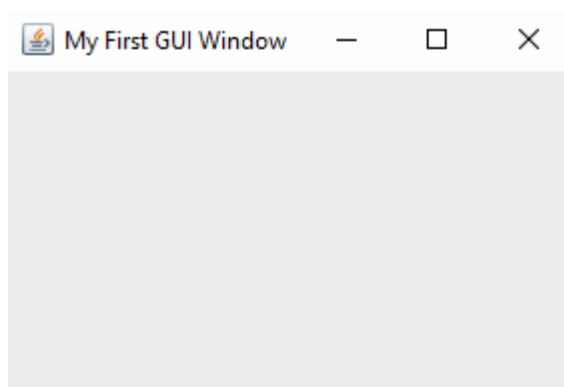
Jaise kisi food delivery app ka main screen hota hai usi tarah yeh main frame hota hai jisme sari cheezein show hoti hain.

### Code:

```
import javax.swing.*;

public class FrameExample {
    public static void main(String[] args) {
        // JFrame ka object banaya jo main window represent karta hai
        JFrame frame = new JFrame("My First GUI Window"); // Window
        // ka title diya
        frame.setSize(300, 200); // Window ka size set kia (width x height)
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); // Jab
        // X button press ho to app band ho jaye
        frame.setVisible(true); // Frame ko visible kia screen par
    }
}
```

### Output:



## Component 2: JLabel Static Text Display

### Purpose:

A **JLabel** shows **text** that the user cannot edit. Useful for titles or instructions.

### Real-Life Example:

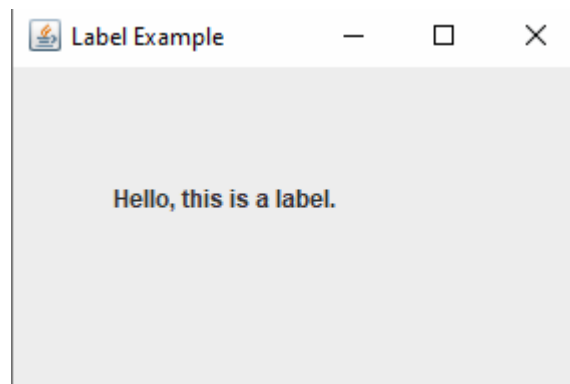
Jaise kisi form mein "**Enter your name**" likha hota hai wo label hota hai.

### Code:

```
import javax.swing.*;

public class LabelExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Label Example");
        frame.setSize(300, 200);
        frame.setLayout(null); // Layout manually set kar rahe hain
        // Label banaya aur usay position par set kia
        JLabel label = new JLabel("Hello, this is a label.");
        label.setBounds(50, 50, 200, 30); // (x, y, width, height)
        frame.add(label); // Frame mein label add kia
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true); // Frame ko show kia
    }
}
```

### Output:



## Component 3: JTextField User Text Input

### Purpose:

A **JTextField** allows the user to **type input** (like name or email).

### Real-Life Example:

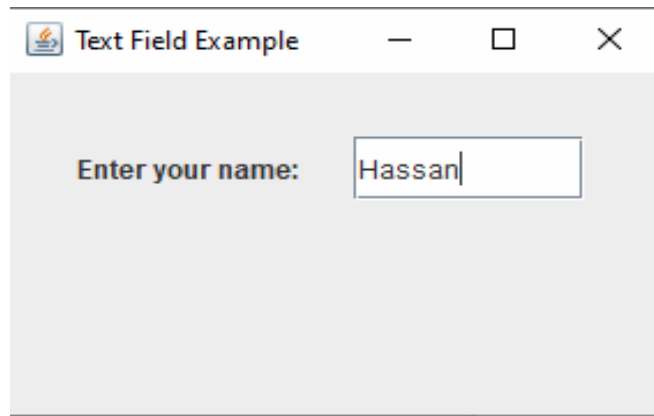
Login form mein jahan user apna naam ya email likhta hai wo text field hoti hai.

### Code:

```
import javax.swing.*;

public class TextFieldExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Text Field Example");
        frame.setSize(300, 200);
        frame.setLayout(null);
        // Label banaya jo user ko guide karega kya input karna hai
        JLabel label = new JLabel("Enter your name:");
        label.setBounds(30, 30, 120, 30);
        frame.add(label);
        // Text field jisme user text enter karega
        JTextField textField = new JTextField();
        textField.setBounds(150, 30, 100, 30); // (x, y, width, height)
        frame.add(textField); // Frame mein label add kia
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

### Output:



## Component 4: JButton Clickable Button

### Purpose:

A **JButton** performs an action when clicked (like Submit or Login).

### Real-Life Example:

Kisi bhi online form mein "**Submit**" ya "**Next**" ka button hota hai wo JButton hota hai.

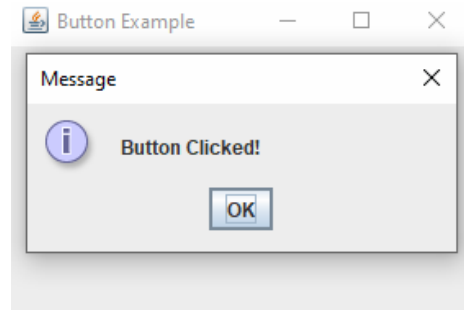
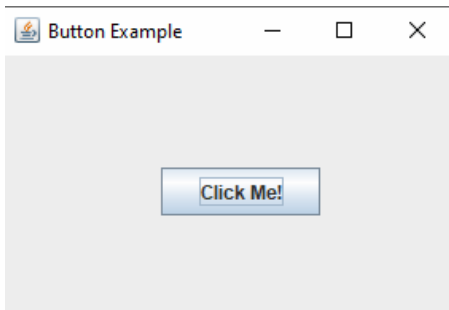
### Code:

```
import javax.swing.*;
import java.awt.event.*;

public class ButtonExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Button Example");
        frame.setSize(300, 200);
        frame.setLayout(null);
        // Button banaya
        JButton button = new JButton("Click Me!");
        button.setBounds(100, 70, 100, 30);
        frame.add(button);
        // Jab button click ho to message show ho
        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                JOptionPane.showMessageDialog(frame, "Button Clicked!");
            }
        });
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

```
}  
}  
}
```

## Output:



## Component 5: JPasswordField Hidden Password Input

### Purpose:

A **JPasswordField** lets the user enter a password that is shown as dots.

### Real Life Example:

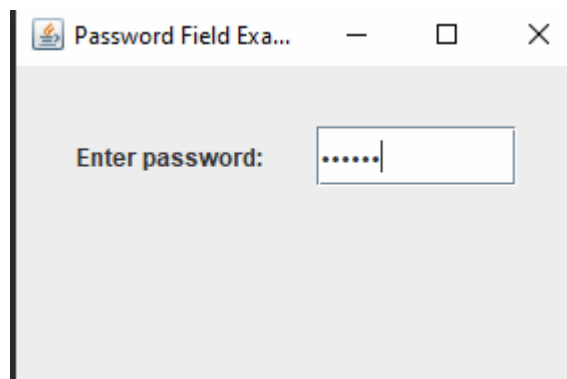
Login screen mein jab password likhte hain to dots ya stars show hoti hain wo password field hoti hai.

### Code :

```
import javax.swing.*;

public class PasswordFieldExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Password Field Example");
        frame.setSize(300, 200);
        frame.setLayout(null);
        // Password label
        JLabel label = new JLabel("Enter password:");
        label.setBounds(30, 30, 120, 30);
        frame.add(label);
        // Password field banaya jisme input hidden hoga
        JPasswordField passField = new JPasswordField();
        passField.setBounds(150, 30, 100, 30);
        passField.setEchoChar('#');
        frame.add(passField);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

### Output:



## Component 6: JOptionPane Popup Box

### Purpose:

**JOptionPane** is used to show a **popup message box** or take quick input.

### Real-Life Example:

Jab app install karte waqt "Installation Successful" ka popup aata hai wo **JOptionPane** se banta hai.

### Code

```
import javax.swing.*;

public class OptionPaneExample {
    public static void main(String[] args) {
        // Simple popup show karega message ke sath
        JOptionPane.showMessageDialog(null, "Welcome to GUI!");
    }
}
```

### Output:

